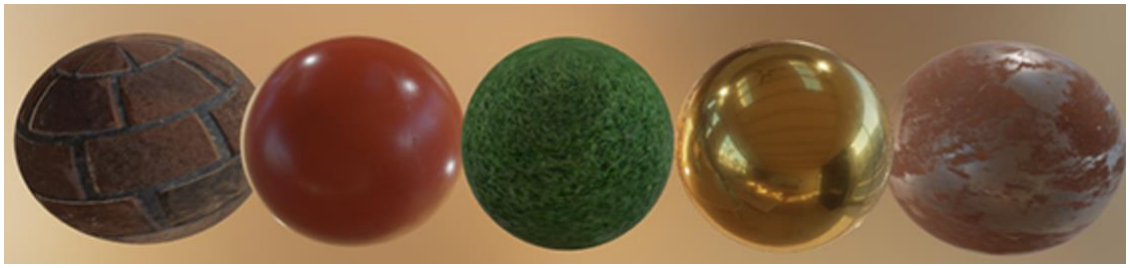
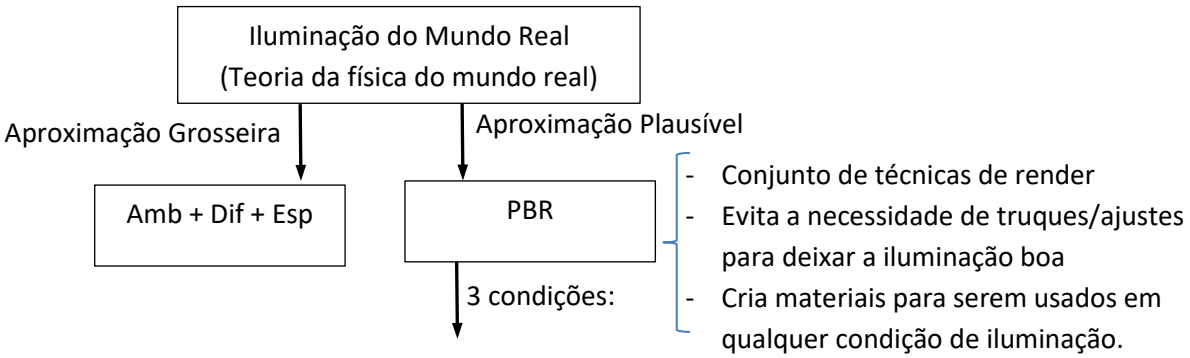
Universidade Federal de Santa Maria
Departamento de Computação Aplicada
Prof. Cesar Tadeu Pozzer
Disciplina: Computação Gráfica Avançada
pozzer@inf.ufsm.br
28/10/2025

Physically-Based Rendering (PBR)

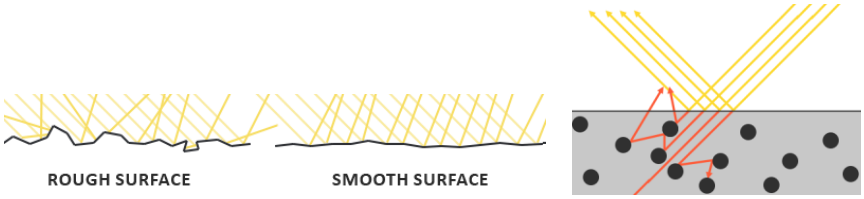
É um conjunto de técnicas para fazer um render realista (próxima aos processos que ocorrem no mundo real). A palavra Based significa que o processo é uma **aproximação da teoria** que melhor descreve os processos físicos do mundo real. Podem-se criar materiais baseados em parâmetros físicos ao invés de se basear em tentativa e erro até gerar algo aceitável. E esses materiais vão se comportar de forma correta independente dos parâmetros de iluminação. Usando PBR não é mais necessário combinar/empilhar diferentes modelos de iluminação, como difusa, especular, reflexões, etc. até que o resultado fique bom, e muitas vezes violando princípios físicos.

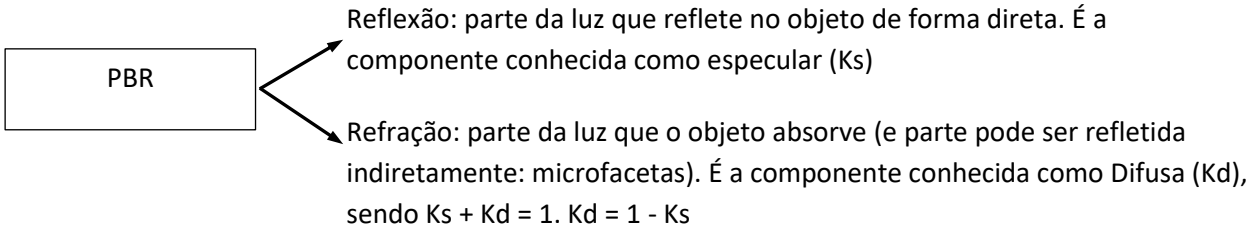


Assim como algoritmos para LOD de terrenos, também existem várias implementações de PBR. Aqui é explorada a versão criada pela Disney (que é baseada em Metallic Workflow) e usada pela Epic Games na **Unreal Engine 4**. Ao final, é apresentado o algoritmo Path Tracing, que também é baseado na rendering equation. Este documento é um apanhado de alguns sites [https://learnopengl.com/PBR/Theory], [https://en.wikipedia.org/wiki/Bidirectional_reflectance_distribution_function]. O site Learnopengl é um ***excelente*** material para aprender OpenGL e tópicos avançados de iluminação e texturização com uso do OpenGL.



1. Baseado em modelo de microfacetas. Tratado pelo BRDF
2. Conservação de energia: Kd e Ks. Luz refletida não pode ser maior que a luz incidente.
3. Usar BRDF – baseado em física. Existem várias implementações





James Kajiya (1986) propôs a Rendering Equation (https://dl.acm.org/doi/pdf/10.1145/15922.15902), uma equação que quando avaliada para cada raio de luz refletido na cena seria capaz de calcular com exatidão a cor final do pixel levando em consideração a contribuição de todos os objetos da cena. Uma explicação visual pode ser observada neste vídeo aqui.

$$I(x,x') = g(x,x')\left[\epsilon(x,x') + \int_S \rho(x,x',x'')I(x',x'')dx''\right]$$

where:

$I(x,x')$	is the related to the intensity of light passing from point x' to point x
$g(x,x')$	is a "geometry" term
$\epsilon(x,x')$	is related to the intensity of emitted light from x' to x
$\rho(x,x'x'')$	is related to the intensity of light scattered from x'' to x by a patch of surface at x'

Rendering Equation: atualmente o melhor modelo
https://en.wikipedia.org/wiki/Rendering_equation

Versão especializada

Reflectance Equation
(usado pelo PBR)

$$\underline{L_o(p,\omega_o)} = \int_\Omega \underline{f_r(p,\omega_i,\omega_o)}\ \underline{L_i(p,\omega_i)}\ \underline{n\cdot\omega_i}\ d\omega_i$$

Produto escalar → Angulo entre normal e luz incidente ω_i → Determina a contribuição difusa.

Radiância L_i sobre o ponto p com ângulo sólido ω_i. Pode ser uma fonte luminosa ou um environment map

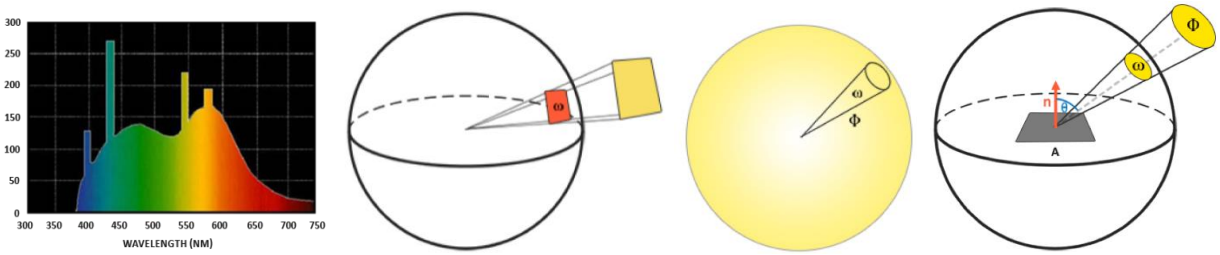
BRDF: escala as intensidades de radiância L_i, assim como o produto escalar, baseado nas propriedades do material (microfacetas), normal e direções da luz.

Soma da Radiance da semiesfera Ω **refletida** na direção ω_o de um ponto p observado de ω_o (direção do observador). A semiesfera está alinhada com a normal da superfície. Na equação, *o* = outgoing, *i* = incoming.

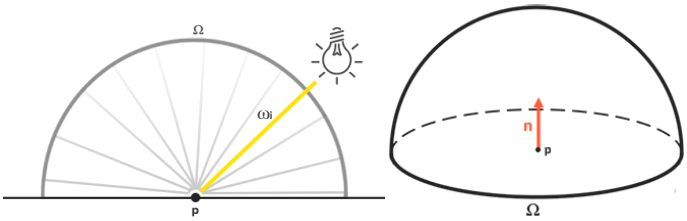
Reflectance Equation: Determina a quantidade de energia L (radiância – luz visível) proveniente de uma única direção. **Reflectância** é a proporção entre o fluxo de radiação eletromagnética incidente numa superfície e o fluxo que é refletido.

1. Radiant Flux Φ: Energia transmitida (em todos os comprimentos de onda – mas na prática avaliada com uma componente RGB) por uma fonte luminosa.
2. Solid Angle: tamanho da área projetada de uma forma sobre uma esfera unitária.
3. Radiant Intensity: Mede a quantidade de Radiant Flux por Solid Angle.

4. Radiance: Energia que chega sobre uma área A em relação a um Solid Angle a partir de uma luz com uma Radiant Intensity. É escalada pelo ângulo de incidência entre a normal e fonte luminosa.



Para uma fonte pontual, a Radiant Intensity da fonte é igual a soma do Radiant Flux de todos os raios (em todas as direções) que partem da fonte. Quando um ponto *p* recebe luz, esta provém de um único vetor ω_i se a fonte for pontual. Se a fonte não for pontual, a Radiância provém de várias direções da mesma fonte, e o cálculo é bem mais complexo. Se existirem várias fontes pontuais, basta somar a radiância de cada uma, como ocorre num processo de iluminação básico já estudado na disciplina de Computação Gráfica.



Como não existe solução numérica para a Reflectance Equation dentro de uma área (ou semiesfera Ω), utiliza-se solução numérica (algorítmica), como mostrado na sequência, que considera todos os raios que chegam nas direções w_i.

$$L_o(p,\omega_o) = \int_\Omega f_r(p,\omega_i,\omega_o)\ L_i(p,\omega_i)\ \ n\cdot\omega_i\ \ d\omega_i$$

```
int steps = 100;
float sum = 0.0f;
vec3 P    = ...;
vec3 Wo   = ...;
vec3 N    = ...;
float dW  = 1.0f / steps;
for(int i = 0; i < steps; ++i)
{
    vec3 Wi = getNextIncomingLightDir(i);
    sum += Fr(P, Wi, Wo) * L(P, Wi) * dot(N, Wi) * dW;
}
```

BRDF (*bidirectional reflectance distribution function*): Trata as propriedades **refletivas** e **refrativas** do material. Tem como entrada a direção da luz ω_i, direção de saída da luz ω_o, normal da superfície e rugosidade (microsurface - microfacetas). Para superfícies perfeitamente polidas, retorna 0 para todos os raios de entrada ω_i que não refletem na direção do observador. Para os que refletem na direção do observador, a função retorna 1.

Existem várias implementações de BRDF [https://en.wikipedia.org/wiki/Bidirectional_reflectance_distribution_function], mas a mais comum é a Cook-Torrance BRDF, que trata tanto a componente especular (Cook-Torrance) como a difusa (Lambert – o ângulo entre normal e luz foi movido para fora da BRDF).

$$f_r = k_d f_{lambert} + k_s f_{cook-torrance}$$

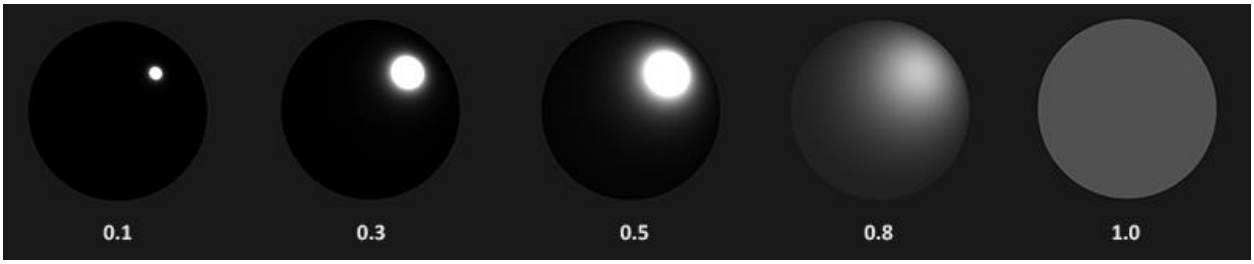
$$f_{lambert} = \frac{c}{\pi}$$

$$f_{cook-torrance} = \frac{DFG}{4(\omega_o \cdot n)(\omega_i \cdot n)}$$

DFG = Normal **D**istribution Function, **F**resnel equation e **G**eometry Function

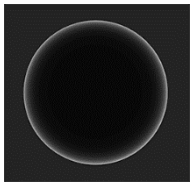
1. **Normal Distribuition Function** (NDF): aproxima a quantidade de microfacetas que estão alinhadas com o halfway vector *h* (vetor médio entre o viewer e a fonte). Existem muitas soluções estatísticas para NDF. Uma delas é a Trowbridge-Reitz GGX. O parâmetro α representa a rugosidade da superfície (ver figura abaixo).

$$NDF_{GGXTR}(n, h, \alpha) = \frac{\alpha^2}{\pi((\mathrm{n} \cdot \mathrm{h})^2(\alpha^2 - 1) + 1)^2}$$



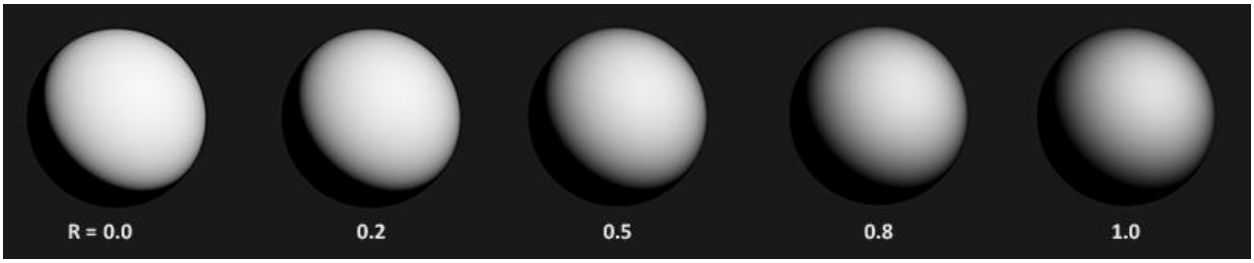
2. **Fresnel equation**: Determina a razão de luz que reflete e que refrata, em função do ângulo que se observa a superfície. A reflexão ocorre mais em ângulos próximos a 90 graus. Fórmula aproximada (Fresnel-Schlick approximation). F_0 representa refletividade da superfície:

$$F_{Schlick}(h, v, F_0) = F_0 + (1 - F_0)\big(1 - (h \cdot \mathrm{v})\big)^5$$



3. **Geometry Function**: Determina estatisticamente a área que as microfacetas causam sombra sobre elas mesmas. Existem várias formulações para isso.

$$G_{SchlickGGX}(n, v, k) = \frac{\mathrm{n} \cdot \mathrm{v}}{(\mathrm{n} \cdot \mathrm{v})(1 - \mathrm{k}) + k}$$



Juntando tudo temos

$$L_o(p, \omega_o) = \int_\Omega f_r(p, \omega_i, \omega_o) \quad L_i(p, \omega_i) \quad n \cdot \omega_i \quad d\omega_i$$

$$L_o(p, \omega_o) = \int_\Omega \left(k_d \frac{c}{\pi} + k_s \frac{DFG}{4(\omega_o \cdot n)(\omega_i \cdot n)}\right) \quad L_i(p, \omega_i) \quad n \cdot \omega_i \quad d\omega_i$$

Como o termo de Fresnel já contém a razão especular (K_s), temos

$$L_o(p, \omega_o) = \int_\Omega \left(k_d \frac{c}{\pi} + \frac{DFG}{4(\omega_o \cdot n)(\omega_i \cdot n)}\right) \quad L_i(p, \omega_i) \quad n \cdot \omega_i \quad d\omega_i$$

Detalhes de implementação prática de PBR em OpenGL com o uso de shaders podem ser vistos em [https://learnopengl.com/PBR/Lighting].

```
float DistributionGGX(vec3 N, vec3 H, float roughness)
{
    float a      = roughness*roughness;
    float a2     = a*a;
    float NdotH  = max(dot(N, H), 0.0);
    float NdotH2 = NdotH*NdotH;

    float num   = a2;
    float denom = (NdotH2 * (a2 - 1.0) + 1.0);
    denom = PI * denom * denom;

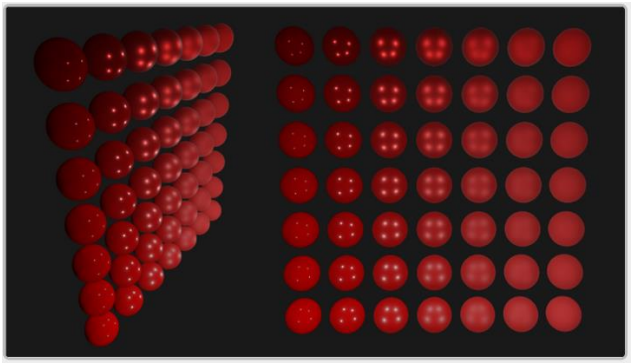
    return num / denom;
}

float GeometrySchlickGGX(float NdotV, float roughness)
{
    float r = (roughness + 1.0);
    float k = (r*r) / 8.0;

    float num   = NdotV;
    float denom = NdotV * (1.0 - k) + k;

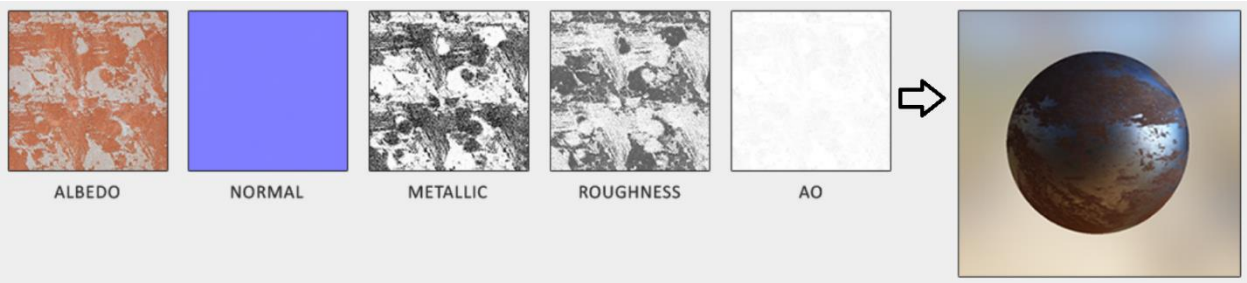
    return num / denom;
}
float GeometrySmith(vec3 N, vec3 V, vec3 L, float roughness)
{
    float NdotV = max(dot(N, V), 0.0);
    float NdotL = max(dot(N, L), 0.0);
    float ggx2  = GeometrySchlickGGX(NdotV, roughness);
    float ggx1  = GeometrySchlickGGX(NdotL, roughness);

    return ggx1 * ggx2;
}
```



Criando materiais PBR

Faz-se uso de texturas para descrever as propriedades físicas de cada material. Com o uso de texturas pode-se especificar como cada pixel vai reagir quando a luz incidir. O ponto pode ser metálico, áspero, rugoso ou ter comportamento específico para diferentes comprimentos de onda da luz.



- Albedo: representa a cor
- Normal: representa o mapa de normais (normal map)
- Metalic: indica se o texel é ou não metálico.
- Roughness: especifica quão rugosa a superfície é. Influencia na orientação das microfacetas.
- AO: representa o ambient occlusion. Indica regiões onde a luz não consegue fugir. Adiciona um fator extra de sombra que agrega muito realismo a cena.